

Data Analysis Programming (Python)

@Ling Liu

针对不完全一致key的dataframe合并

在数据合并的过程中，我们通常会以两个表格中共同的一栏来作为合并的key，而后将其他栏合并。比如，两张表格分别是同一个班同学的两个不同选课的成绩。那么这个班里同学的姓名或者学号就可以看成是这个用以合并的key，也就是我们根据同学的学号来对应每位同学的两门的成绩。

但是在一些情况下，可能会出现这个合并key的内容不完全一致，比如都是城市名，可能第一个表格用的是“北京、上海。。”等写法，但是另外一个表格则是采用“北京市、上海市。。”的写法，从意思上来看，二者很明显有对应关系，但是对程序而言，北京与北京市是完全不同的两个值。

因此，本notebook就讲解一下遇到类似问题的处理方法

```
import pandas as pd
```

首先，我们读入两个表格，这两个表格都是与城市相关的数据，我们需要把两个数据以城市为主key合并到一起。

```
df1 = pd.read_excel(r'中国城市数据库.xlsx')
df2 = pd.read_excel(r'2003-2021地级市当年实际使用外资金额.xlsx')
```

```
df1.head()
```

	年份	行政区划代码	地区	地区生产总值(万元)	人均地区生产总值(元)	第二产业从业人员比重(%)	人口密度(人/平方公里)	地区生产总值增长率(%)	固定资产投资总额(万元)	地方财政一般预算内支出(万元)	城市代码	CityCode
0	2022.0	110000.0	北京	416110000.0	190313.0	NaN	NaN	0.7	NaN	74691549.0	1100.0	110000.0
1	2021.0	110000.0	北京	402700000.0	183980.0	NaN	NaN	8.5	NaN	72051201.0	1100.0	110000.0
2	2020.0	110000.0	北京	361030000.0	164889.0	NaN	NaN	1.2	NaN	71161762.0	1100.0	110000.0
3	2019.0	110000.0	北京	353710000.0	164220.0	15.57	NaN	6.1	NaN	74082503.0	1100.0	110000.0
4	2018.0	110000.0	北京	303199787.0	140211.0	16.55	NaN	6.6	NaN	74675240.0	1100.0	110000.0

```
df2.head()
```

	编号	省份代码	城市代码	省份	城市	年份	当年实际使用外资金额 (万美元)	美元/人民币年均汇率 (1美元：元)	当年实际使用外资金额 (元)
0	1.0	11	1100	北京市	北京市	2003	214675	8.277	177.686498
1	2.0	12	1200	天津市	天津市	2003	163325	8.277	135.184102
2	3.0	13	1301	河北省	石家庄市	2003	25414	8.277	21.035168
3	4.0	13	1302	河北省	唐山市	2003	20198	8.277	16.717885
4	5.0	13	1303	河北省	秦皇岛市	2003	15443	8.277	12.782171

但是通过上面导入数据后，我们可以看到df1中，城市名为 北京，但是df2中为北京市。此外，观察其他相关栏，我们可以看到，df1中存在一个行政区划代码，而df2中则为，省份代码和城市代码两栏。再仔细分析，可以猜测，其中df2中的城市代码前两位与省份代码相同，同时，df2中的4位城市代码与df1中行政区划代码的前四位也相同。（请注意，在上面的代码中，我们只print了每个df的前5行，因此前面的猜测不一定是完全准确的。实际操作过程中必须要经过仔细地查找与核对，确定对应栏的对应内容是一致的才可以继续进行合并。）

那么在这个情况下，意味着我们有两种可行的方法：

1. 把df2中的北京市的“市”去掉，或者把df1中城市名后面加上“市”，以保证二者一致
2. 提取df1中的行政区划代码中前四位单独建立新的一栏作为城市代码，这样就保证与df2中的城市代码一致

接下来，我们以第二种方法为例，讲解该如何完成。

首先我们要确定两个代码，df1中的行政区划代码和df2中的城市代码数据类型，如果数据类型不同，即便值看起来一样，也无法一一对应。

```
# 查看df1,df2各栏dtype
print(df1.dtypes)
print(df2.dtypes)
```

年份	float64
行政区划代码	float64
地区	object
地区生产总值(万元)	float64
人均地区生产总值(元)	float64
第二产业从业人员比重(%)	float64
人口密度(人 / 平方公里)	float64
地区生产总值增长率(%)	float64
固定资产投资总额(万元)	float64
地方财政一般预算内支出(万元)	float64

```
dtype: object
编号                float64
省份代码            int64
城市代码            int64
省份                object
城市                object
年份                int64
当年实际使用外资金额 ( 万美元)    object
美元/人民币年均汇率 ( 1美元 : 元 )    float64
当年实际使用外资金额 ( 元 )        float64
dtype: object
```

可以看出两者一个是int, 一个是float.虽然类型不完全一致，但是都是数值类型，因此保证二者值相等即可。接下来对df1的行政代码做处理。

此处依然有两种方法：

- 1. 通过数值计算保留前4位，即110000/100直接得到前四位
- 2. 把该栏数值转换为字符串，而后去除前四位

我们分别尝试一下

```
# 直接对数值进行处理
df1['行政区划代码']/100
```

```
0      1100.0
1      1100.0
2      1100.0
3      1100.0
4      1100.0
...
9896   6505.0
9897   6505.0
9898   6505.0
9899   6505.0
9900      NaN
Name: 行政区划代码, Length: 9901, dtype: float64
```

可以看到，受益于pandas对数值的整体处理，我们直接得到了全部行对应此栏的“前四位”，为了不影响原始数据，我们通常会赋值给新的一栏

```
df1['城市代码'] = df1['行政区划代码']/100
df1.head()
```

	年份	行政区划代码	地区	地区生产总值(万元)	人均地区生产总值(元)	第二产业从业人员比重(%)	人口密度(人 / 平方公里)	地区生产总值增长率(%)	固定资产投资总额(万元)	地方财政一般预算内支出(万元)	城市代码
0	2022.0	110000.0	北京	416110000.0	190313.0	NaN	NaN	0.7	NaN	74691549.0	1100.0

	年份	行政区划代码	地区	地区生产总值(万元)	人均地区生产总值(元)	第二产业从业人员比重(%)	人口密度(人 / 平方公里)	地区生产总值增长率(%)	固定资产投资总额(万元)	地方财政一般公共预算内支出(万元)	城市代码
1	2021.0	110000.0	北京	402700000.0	183980.0	NaN	NaN	8.5	NaN	72051201.0	1100.0
2	2020.0	110000.0	北京	361030000.0	164889.0	NaN	NaN	1.2	NaN	71161762.0	1100.0
3	2019.0	110000.0	北京	353710000.0	164220.0	15.57	NaN	6.1	NaN	74082503.0	1100.0
4	2018.0	110000.0	北京	303199787.0	140211.0	16.55	NaN	6.6	NaN	74675240.0	1100.0

第二种方法，先将行政区域划分代码，转成string，而后取值，再转换数据类型为int

```
# astype可以用来对df做数据类型转换
df1['行政区划代码'].astype(str)
```

```
0      110000.0
1      110000.0
2      110000.0
3      110000.0
4      110000.0
...
9896    650500.0
9897    650500.0
9898    650500.0
9899    650500.0
9900         nan
Name: 行政区划代码, Length: 9901, dtype: object
```

```
# 为了后续方便，我们转换之后，赋值给新的一栏
df1['CityCode'] = df1['行政区划代码'].astype(str)
```

pandas提供了对整栏和整列字符串数值做整体操作的方法

详细介绍及介绍见：https://pandas.pydata.org/pandas-docs/stable/user_guide/text.html

简单一句话，如果df里面某一栏的值都是字符串，那么可以直接对整行进行操作

```
series.str.lower()
```

支持一切python自带的字符串操作

除去python自带的字符串操作外，pandas还额外添加了如下的方法：

```
get()
slice()
slice_replace()
cat()
repeat()
normalize()
pad()
wrap()
join()
get_dummies()
```

pandas提供了对整栏和整列字符串数值做整体操作的方法，具体操作如下：

```
df1.CityCode.str.slice(0,4)
```

```
0      1100
1      1100
2      1100
3      1100
4      1100
...
9896   6505
9897   6505
9898   6505
9899   6505
9900    nan
Name: CityCode, Length: 9901, dtype: object
```

而后我们再按照前面介绍过的数据类型转换的方式，将此栏转为int即可。